

DCBNet

Setup & Run Guide — Step-by-step instructions for Kaggle and local environments

Documentation for reproducing training and evaluation of DCBNet

Contents

1	Getting the Code	3
2	Getting the Datasets	3
2.1	LEVIR-CD (Public — available on Kaggle)	3
2.2	LIM-CD (Private Access / Official GitHub)	3
3	Running on Kaggle	4
3.1	Step 1 — Create a Kaggle Account	4
3.2	Step 2 — Create a New Notebook and Import the Code File	4
3.3	Step 3 — Select a GPU Accelerator	6
3.4	Step 4 — Add Datasets as Inputs	6
3.5	Step 5 — Configuration Cell Setup	7
3.5.1	Mode Selection	7
3.5.2	Weight Paths (Evaluation Modes Only)	8
3.5.3	Epoch and Unfreeze Epoch Settings	8
3.5.4	Dataset Selection	9
3.6	Step 6 — Running the Notebook	10
3.6.1	Option A — Interactive Session (Quick Tests)	10
3.6.2	Option B — Background Session (Full Training Runs)	10
4	Running on a Local Machine	11
4.1	Step 1 — Install Dependencies	11
4.2	Step 2 — Download and Organise Datasets	12
4.3	Step 3 — Set Local Paths in the Configuration Cell	12
4.4	Step 4 — Run the Notebook	12
5	Running the Gradio Web Interface	13
5.1	Step 1 — Download the Web Application	13

5.2	Step 2 — Set Up a Dedicated Environment	13
5.2.1	Create and activate the environment	14
5.2.2	Install required packages	14
5.3	Step 3 — Launch the Application	14
6	Quick Reference Card	15

1 Getting the Code

The notebook file (`.ipynb`) is hosted and can be obtained from one of the two sources below. You only need to download it once.

Source	Link / Instructions
GitHub	github.com/BotRavat/urban-change-detection-DCBNet Click the green Code button → Download ZIP , then extract the archive. The notebook file ends in <code>.ipynb</code> .
Google Drive	drive.google.com/drive/folders/13uSoW1... Open the link, right-click the file, and select Download . The <code>.ipynb</code> notebook will save to your Downloads folder.

Note: Save the downloaded `.ipynb` file somewhere easy to find, such as your Desktop or Downloads folder. You will upload it to Kaggle in the next step.

2 Getting the Datasets

Two datasets are used in this project. Follow the instructions for the one you need.

2.1 LEVIR-CD (Public — available on Kaggle)

1. Go to <https://www.kaggle.com> and search for **LEVIR-CD**.
2. Open the dataset page and click **Download** (you need a free Kaggle account).
3. You do not need to manually download it if you are running on Kaggle — it can be added directly from the notebook's input section (covered in Section 3.4).

2.2 LIM-CD (Private Access / Official GitHub)

LIM-CD is not publicly listed on Kaggle. You have two options:

1. **Request access from us:** Send us an email or message with your kaggle username and we will grant you access to the private Kaggle dataset. Once access is granted, it will appear in your Kaggle account and can be added as an input (same steps as Section 3.4).
2. **Download from the official repository:**
<https://github.com/xiaoxiangAQ/LIM-CD-dataset>
Download the dataset from there and upload it manually as a Kaggle dataset or place it in the correct folder if running locally.

Contact for LIM-CD access:

7sravat@gmail.com

3 Running on Kaggle

Kaggle is a free online platform that gives you access to GPUs without needing to install anything. All steps below happen inside your web browser.

3.1 Step 1 — Create a Kaggle Account

If you do not already have one, go to <https://www.kaggle.com> and sign up for a free account. Email verification is required.

3.2 Step 2 — Create a New Notebook and Import the Code File

1. From the Kaggle home page, click **Code** in the left sidebar.
2. Click the blue **New Notebook** button (top right).
3. A blank notebook will open. In the top menu, click **File** → **Import Notebook**.
4. A dialog will appear. Click **Browse** and select the `.ipynb` file you downloaded in Section 1.
5. Click **Import**. The notebook with all the code will load.

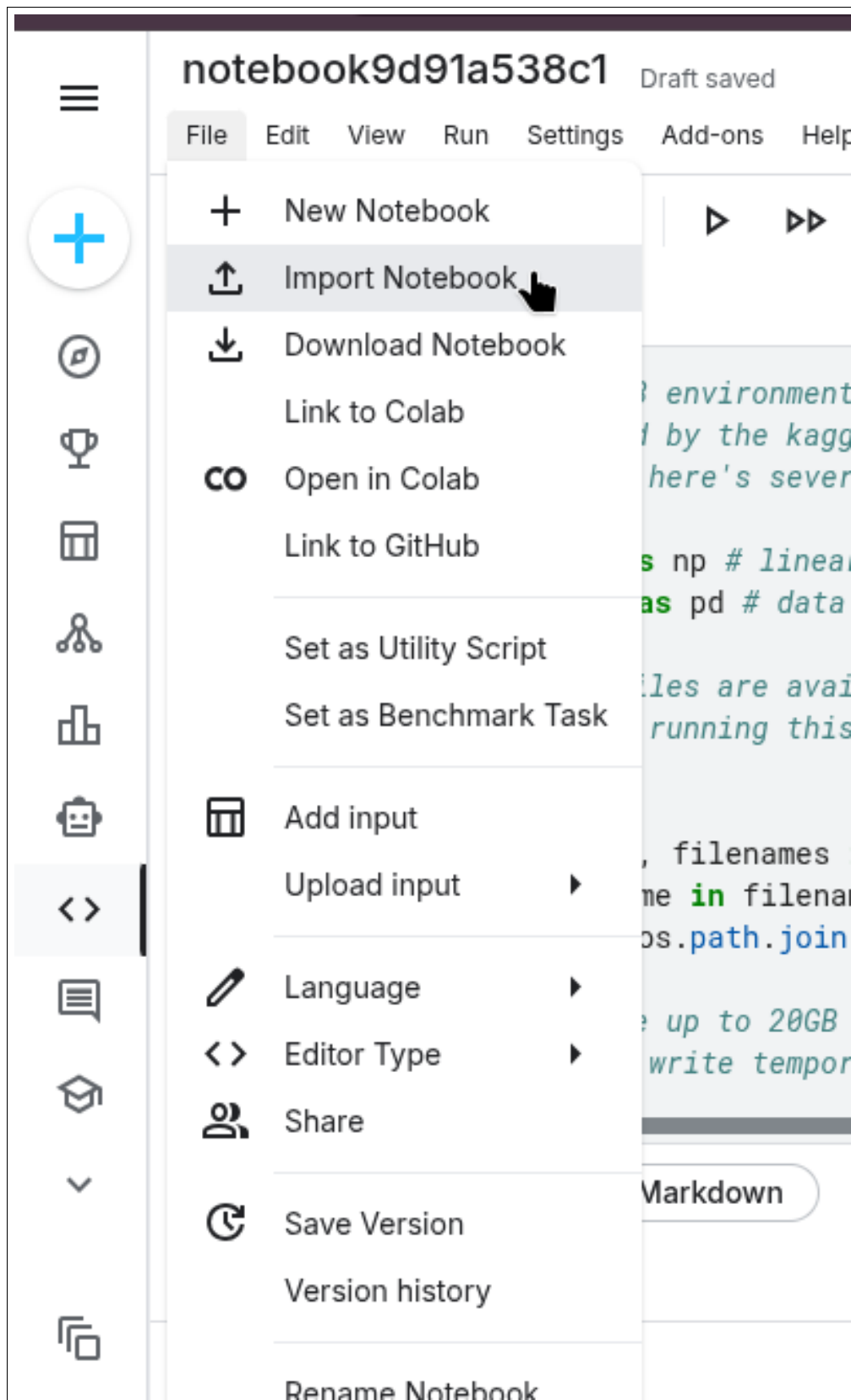


Figure 1: Importing the notebook file into Kaggle. Click File → Import Notebook, then browse to your downloaded .ipynb file.

3.3 Step 3 — Select a GPU Accelerator

The model training step requires a GPU. Kaggle provides two free options: **T4 GPU** and **P100 GPU**. Either works fine.

1. On the right side of the notebook page, find the **Accelerator** section (it may be inside a **Settings** panel — look for a gear icon or “Session options”).
2. Click the dropdown and select either **GPU T4 x1** or **GPU P100**.
3. The page will briefly refresh to activate the GPU session.

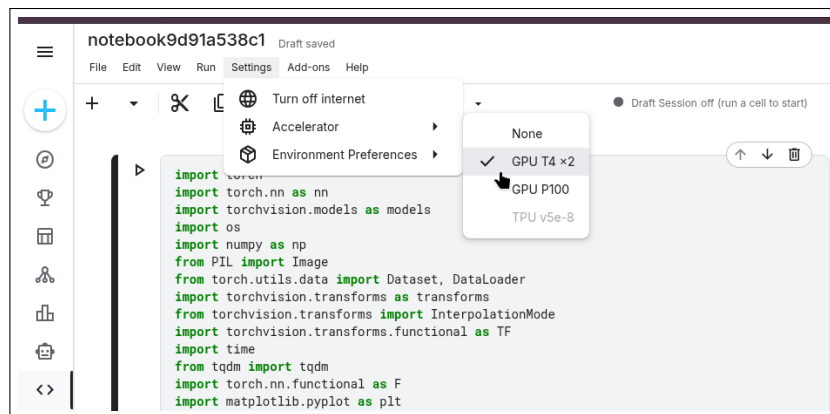


Figure 2: GPU selection panel on Kaggle. Choose either T4 or P100 from the Accelerator dropdown on the right-hand settings panel.

Important: Kaggle gives 30 hours of free GPU time per week. If training on both datasets, it is recommended to use the background session option described in Section 3.6 to avoid losing progress if the browser is closed.

3.4 Step 4 — Add Datasets as Inputs

1. On the right panel, find the **Input** section and click **Add Input** (or the + icon next to it).
2. A search dialog will appear. Type **LEVIR-CD** in the search bar and press Enter.
3. Click on the correct result and press **Add button (+)**. The dataset will be mounted automatically and available at a path like:
`/kaggle/input/levir-cd/`
4. For **LIM-CD**, repeat the same steps if you have been granted access (the dataset will appear in your search results). If not, upload it manually via **New Dataset** under your Kaggle profile and then add it as input.

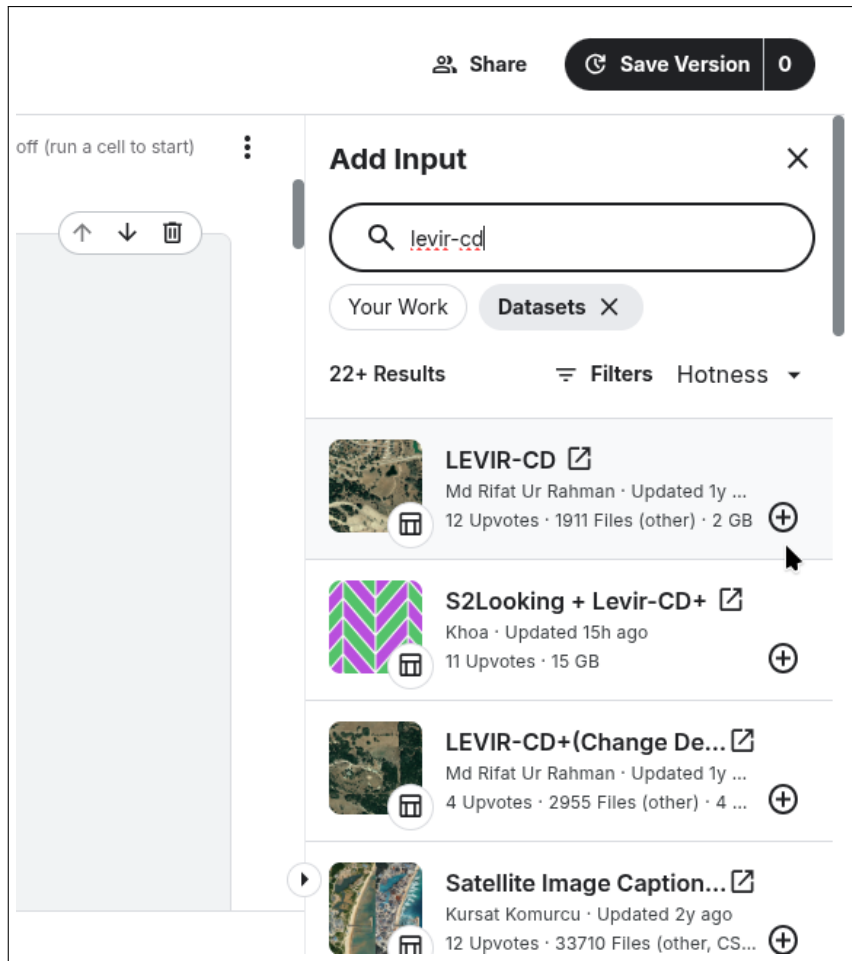


Figure 3: Adding a dataset in Kaggle. Click the + icon in the Input panel, search for the dataset name, and click Add.

If you want to evaluate only (not train from scratch), you also need to upload your saved model weights as a Kaggle dataset or as an input file, then add it here the same way. See Section 3.5 for details.

3.5 Step 5 — Configuration Cell Setup

Near the top of the notebook there is a clearly marked **Configuration Cell**. This is the only cell you need to edit. All key settings are collected here so you do not need to search through the rest of the code.

3.5.1 Mode Selection

Set the **MODE** variable to one of the options in Table 1.

Table 1: Available run modes and their descriptions

Mode Value	Description
"s1only"	Train Stage 1 only (pixel-level change detection)
"s2only"	Train Stage 2 only (object-level refinement)
"both"	Train Stage 1 followed by Stage 2 sequentially
"eval_s1"	Evaluate Stage 1 model only — requires weight path
"eval_s2"	Evaluate Stage 2 model only — requires weight path
"eval_all"	Evaluate both stages together — requires both weight paths

Example:

```
MODE = "both"    # change this to any value from the table above
```

3.5.2 Weight Paths (Evaluation Modes Only)

If you selected any `eval_*` mode, you must provide the paths to the saved model weights. Upload the weight files (`.pth`) to the Kaggle Input section (same way as datasets), then set:

```
S1_WEIGHTS = "/kaggle/input/your-weights-dataset/stage1_best.pth"
S2_WEIGHTS = "/kaggle/input/your-weights-dataset/stage2_best.pth"
```

If you are in a training mode, these paths are ignored and you can leave them as empty strings (`""`).

3.5.3 Epoch and Unfreeze Epoch Settings

```
# Stage 1 settings
S1_NUM_EPOCHS      = 100    # total training epochs for Stage 1
S1_UNFREEZE_EPOCH  = 5      # epoch at which backbone unfreezes in
                             Stage 1

# Stage 2 settings
S2_NUM_EPOCHS      = 50     # total training epochs for Stage 2
S2_UNFREEZE_EPOCH  = 5      # epoch at which backbone unfreezes in
                             Stage 2
```


The values shown above are what we used when training our submitted model. You may use these exact values or set your own. `UNFREEZE_EPOCH` controls when the backbone encoder switches from frozen to trainable — it should always be set to a value **well below** the total epoch count so the model has enough remaining epochs to fine-tune after unfreezing.

Keep `UNFREEZE_EPOCH` smaller than `NUM_EPOCHS`.

For example, with `NUM_EPOCHS` = 90, setting `UNFREEZE_EPOCH` = 85 would leave almost no time for the backbone to fine-tune, which defeats its purpose. A gap of at least 30–40 epochs between unfreeze and end of training is recommended.

Recommended defaults (what we used):

LEVIR-CD:

`S1_NUM_EPOCHS` = 90–95 `S1_UNFREEZE_EPOCH` = 15–20
`S2_NUM_EPOCHS` = 40–45 `S2_UNFREEZE_EPOCH` = 15–20

LIM-CD:

`S1_NUM_EPOCHS` = 45–50 `S1_UNFREEZE_EPOCH` = 15–20
`S2_NUM_EPOCHS` = 40–45 `S2_UNFREEZE_EPOCH` = 15–20

These serve as safe starting points. LIM-CD requires fewer epochs overall as the dataset converges faster than LEVIR-CD. Stage 2 in both cases needs fewer epochs because it builds on already-trained Stage 1 features rather than learning from scratch.

3.5.4 Dataset Selection

```
DATASET = "LEVIR-CD"    # options: "LEVIR-CD"    or    "LIM-CD"
```

Only these two values are currently supported. Make sure the corresponding dataset has been added as an input (Section 3.4) before running.

You must also set the correct root path for the chosen dataset. On Kaggle, the safest way to get the exact path is to copy it directly from the **Input** panel:

1. After adding the dataset in the Input section, hover over its name in the panel — a small **copy path** icon or the full path string will appear beneath it.
2. Copy that path (it will look something like
`/kaggle/input/levir-cd/`

or

`/kaggle/input/lim-cd-dataset/`).

3. Paste it into the corresponding entry in the configuration cell:

```
DATASET_ROOT = {  
    "LEVIR-CD" : "/kaggle/input/levir-cd/",          # paste your  
                copied path here  
    "LIM-CD"   : "/kaggle/input/lim-cd-dataset/",    # paste your  
                copied path here  
}
```

Local machine users: Path configuration for local runs is covered separately in Section 4.3 . The DATASET_ROOT entries above are specifically for the Kaggle environment.

3.6 Step 6 — Running the Notebook

You have two options for how to run the notebook on Kaggle.

3.6.1 Option A — Interactive Session (Quick Tests)

Use this when you want to run cells one by one or test something quickly.

1. Click **Run All** from the top menu (**Run** → **Run All Cells**).
2. The output will appear below each cell as it executes.
3. **Do not close the browser tab** while running — the session will stop if the tab is closed or the connection drops.

Interactive sessions time out after approximately **9 hours** of inactivity. For full training runs, use Option B below.

3.6.2 Option B — Background Session (Full Training Runs)

Background sessions (called **Save Version** on Kaggle) run even after you close the browser. This is the recommended option for full training.

1. Click the **Save Version** button at the top right of the notebook page.
2. A dialog will appear with two options — select **Save & Run All (Commit)**.
3. Under **Advanced Settings**, ensure:
 - **Environment** is set to the same GPU you selected.
 - **Save Output** is enabled so results and saved models are stored.

4. Click **Save**. Kaggle will queue and run the notebook in the background.
5. You can close the browser. To check progress, go to your Kaggle profile → **Code** and open the notebook — it will show a running indicator.

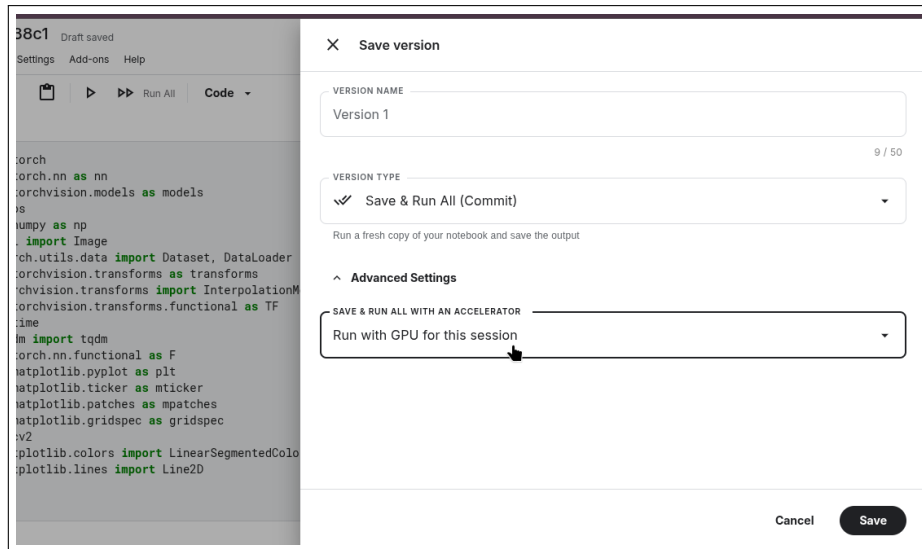


Figure 4: Save Version dialog on Kaggle. Select “Save & Run All (Commit)” to start a background session that continues even after closing the browser.

Output files (trained model weights, plots, logs) are saved automatically under `/kaggle/working/` and accessible from the notebook’s Output tab after the run completes.

4 Running on a Local Machine

If you prefer to run the code on your own computer rather than Kaggle, follow the steps below. A machine with an NVIDIA GPU is strongly recommended for training; evaluation can be done on CPU but will be slow.

4.1 Step 1 — Install Dependencies

At the very top of the notebook there is a dedicated cell containing all required `pip install` commands. Run this cell first before anything else

4.2 Step 2 — Download and Organise Datasets

Place your downloaded datasets in any folder on your computer. The recommended folder structure is:

```
project_folder/
  notebook.ipynb
  datasets/
    LEVIR-CD/
      train/
        A/          <- pre-change images
        B/          <- post-change images
        label/      <- binary change masks
      val/
      test/
    LIM-CD/
      train/
      val/
      test/
```

4.3 Step 3 — Set Local Paths in the Configuration Cell

In the configuration cell, update the dataset root paths to point to your local folders:

```
# For local runs: update these paths
DATASET_ROOT = {
    "LEVIR-CD" : "/path/to/your/datasets/LEVIR-CD",
    "LIM-CD"   : "/path/to/your/datasets/LIM-CD",
}

# For local weight loading (eval modes):
S1_WEIGHTS = "/path/to/your/weights/stage1_best.pth"
S2_WEIGHTS = "/path/to/your/weights/stage2_best.pth"
```

Replace `/path/to/your/...` with the actual folder path on your machine. On Windows, use forward slashes or raw strings, for example:

```
r"C:\Users\yourname\datasets\LEVIR-CD"
```

4.4 Step 4 — Run the Notebook

Open the notebook in JupyterLab or Jupyter Notebook:

```
jupyter notebook notebook.ipynb
```

Then click **Kernel** → **Restart & Run All** to execute all cells from top to bottom. All outputs, plots, and saved weights will appear in the same folder as the notebook.

5 Running the Gradio Web Interface

The project includes a ready-to-use web interface built with [Gradio](#) that lets you run change detection on your own image pairs without writing any code. The interface loads pre-trained weights for both stages and both datasets (four model files total) and produces a visual output directly in the browser.

5.1 Step 1 — Download the Web Application

The web folder can be obtained from one of the two sources below.

Source	Link / Instructions
Google Drive	drive.google.com/drive/folders/13uSoW1... Download the entire <code>web/</code> folder. This version includes all four pre-trained model weight files (<code>levir_s1.pth</code> , <code>levir_s2.pth</code> , <code>lim_s1.pth</code> , <code>lim_s2.pth</code>) so the interface runs immediately without any extra setup.
GitHub	github.com/BotRavat/urban-change-detection-DCBNet- The repository contains the code only. After downloading, you must separately download the four <code>.pth</code> weight files from the Google Drive link above and place them in the root of the <code>web/</code> folder before running.

After downloading, the `web/` folder should have the following structure:

```
web/  
  app.py  
  group.py  
  interface.html  
  levir_s1.pth      <- LEVIR-CD Stage 1 weights  
  levir_s2.pth      <- LEVIR-CD Stage 2 weights  
  lim_s1.pth        <- LIM-CD Stage 1 weights  
  lim_s2.pth        <- LIM-CD Stage 2 weights
```

All four `.pth` files must be present in the same folder as `app.py` before launching. If any weight file is missing, the application will raise a `FileNotFoundError` on startup.

5.2 Step 2 — Set Up a Dedicated Environment

It is strongly recommended to run the web interface inside a dedicated Python virtual environment to avoid conflicts with other packages on your system.

5.2.1 Create and activate the environment

On macOS / Linux:

```
python -m venv dcbnet-env
source dcbnet-env/bin/activate
```

On Windows (Command Prompt):

```
python -m venv dcbnet-env
dcbnet-env\Scripts\activate
```

Your terminal prompt will show (dcbnet-env) when the environment is active.

5.2.2 Install required packages

With the environment active, run the following in your terminal or Command Prompt:

```
pip install torch torchvision numpy opencv-python pillow gradio
```

5.3 Step 3 — Launch the Application

1. Open a terminal (or Command Prompt on Windows).
2. Activate the virtual environment if it is not already active (see Step 2 above).
3. Navigate to the root of the `web/` folder:

```
cd /path/to/web
```

Then launch the app:

```
python app.py
```

4. Gradio will start a local server. You will see output similar to:

```
Running on local URL:  http://127.0.0.1:7860
```

5. Open your web browser and go to: `http://127.0.0.1:7860`
The interface will load and is ready to use.
6. To stop the application, go back to the terminal and press **Ctrl + C**.

The interface is self-contained. Once running, you can upload a pre-change image and a post-change image, select the dataset model (LEVIR-CD or LIM-CD), and the application will return the detected change map directly in the browser. No additional configuration is needed.

6 Quick Reference Card

Table 2: At-a-glance checklist before pressing Run

#	Step	What to check
1	Notebook imported	.ipynb file loaded in Kaggle or Jupyter
2	GPU selected	T4 or P100 chosen in Accelerator panel
3	Dataset added	LEVIR-CD or LIM-CD visible in Input panel
4	MODE set	One of the six valid mode strings
5	DATASET set	"LEVIR-CD" or "LIM-CD" only
6	Weights provided	Only needed for <code>eval_*</code> modes
7	Epochs configured	Default 100 / 20, or your own values
8	Session type chosen	Interactive (quick) or Save Version (full run)

You are ready to run.

If anything is unclear or you encounter an error not covered here, contact us at 7sravat@gmail.com.